Large and Fast: Exploiting Memory Hierarchy: Part II

CSE 4305: Computer Organization and Architecture

Mohammad Ishrak Abedin
Lecturer, Department of CSE

09 February 2026

Islamic University of Technology

Table of Contents

Virtual Machine and Virtual Memory
A Common Framework for Memory Hierarchy
Using a Finite-State Machine to Control a Simple Cache
Parallelism and Memory Hierarchy: Cache Coherence
Conclusion

. . . a system has been devised to make the core drum combination appear to the programmer as a single level store, the requisite transfers taking place automatically.

One-level storage system, 1962

Kilburn et al.

Virtual Machine and Virtual Memory

Virtual Machines (VMs)

- Provide a complete system-level environment at the binary instruction set architecture level

Virtual Machines (VMs)

- Provide a complete system-level environment at the binary instruction set architecture level
- Present the illusion that users have an entire computer to themselves, including a copy of the operating system

Virtual Machines (VMs)

- Provide a complete system-level environment at the binary instruction set architecture level
- Present the illusion that users have an entire computer to themselves, including a copy of the operating system
- The software that supports VMs is called a Virtual Machine Monitor (VMM) or Hypervisor

Virtual Machines (VMs)

- Provide a complete system-level environment at the binary instruction set architecture level
- Present the illusion that users have an entire computer to themselves, including a copy of the operating system
- The software that supports VMs is called a Virtual Machine Monitor (VMM) or Hypervisor
- The VMM manages the host hardware while sharing resources among the guest VMs

Virtual Machines (VMs)

- Provide a complete system-level environment at the binary instruction set architecture level
- Present the illusion that users have an entire computer to themselves, including a copy of the operating system
- The software that supports VMs is called a Virtual Machine Monitor (VMM) or Hypervisor
- The VMM manages the host hardware while sharing resources among the guest VMs
- To control privileged state and I/O, the VMM must run at a higher privilege level than the guest VM

Why Use Virtual Machines?

- Allow the sharing of a single computer among many unrelated users, which is the foundation of Cloud computing

Why Use Virtual Machines?

- Allow the sharing of a single computer among many unrelated users, which is the foundation of Cloud computing
- Improve dependability by isolating systems and protecting users from each other

Why Use Virtual Machines?

- Allow the sharing of a single computer among many unrelated users, which is the foundation of Cloud computing
- Improve dependability by isolating systems and protecting users from each other
- Simplify software management by allowing a single server to run multiple software stacks, including legacy OSes

Why Use Virtual Machines?

- Allow the sharing of a single computer among many unrelated users, which is the foundation of Cloud computing
- Improve dependability by isolating systems and protecting users from each other
- Simplify software management by allowing a single server to run multiple software stacks, including legacy OSes
- Improve hardware management by consolidating servers and supporting the migration of running VMs to balance loads

Virtual Memory

- Acts as a cache by using main memory to temporarily hold active data from secondary storage like disks or flash

Virtual Memory

- Acts as a cache by using main memory to temporarily hold active data from secondary storage like disks or flash
- Enables efficient and safe sharing of memory among several programs or virtual machines

Virtual Memory

- Acts as a cache by using main memory to temporarily hold active data from secondary storage like disks or flash
- Enables efficient and safe sharing of memory among several programs or virtual machines
- Relieves programmers from the historical burden of manually making large programs fit into limited main memory

Virtual Memory

- Acts as a cache by using main memory to temporarily hold active data from secondary storage like disks or flash
- Enables efficient and safe sharing of memory among several programs or virtual machines
- Relieves programmers from the historical burden of manually making large programs fit into limited main memory
- Implements the translation of a program's virtual address space to physical addresses

Paging and Translation

- A virtual memory block is called a page and a virtual memory miss is called a page fault

Paging and Translation

- A virtual memory block is called a page and a virtual memory miss is called a page fault
- The processor generates a virtual address, which is broken into a virtual page number and a page offset

Paging and Translation

- A virtual memory block is called a page and a virtual memory miss is called a page fault
- The processor generates a virtual address, which is broken into a virtual page number and a page offset
- The virtual page number translates to a physical page number, while the page offset remains unchanged

Paging and Translation

The processor does not directly request data from storage when it generates an address; instead, it generates a virtual address that represents the location of data in its own logical (virtual) address space. This address is first translated by the system (using the page table or TLB) into a **physical address** that points to a location in main memory (RAM). If the required page is already present in RAM, the data is accessed immediately and quickly. However, if the page is not in RAM (a situation called a **page fault**), only then does the operating system step in to fetch the required page from secondary storage (disk) and load it into a physical page in RAM. After this, the translation is updated and the access proceeds. So, the CPU always targets memory via virtual addresses, and disk access happens only as a fallback when the needed data is not already in main memory.

- A virtual memory block is called a page and a virtual memory miss is called a page fault
- The processor generates a virtual address, which is broken into a virtual page number and a page offset
- The virtual page number translates to a physical page number, while the page offset remains unchanged
- Having more virtual pages than physical pages creates the illusion of a much larger amount of memory

Handling Page Faults and Page Tables

- A page fault takes millions of clock cycles, so systems use fully associative placement to reduce fault frequency

Handling Page Faults and Page Tables

- A page fault takes millions of clock cycles, so systems use fully associative placement to reduce fault frequency
- A page table resides in main memory to locate these pages without impractical full searches

Handling Page Faults and Page Tables

- A page fault takes millions of clock cycles, so systems use fully associative placement to reduce fault frequency
- A page table resides in main memory to locate these pages without impractical full searches
- The page table is indexed by the virtual page number to discover the corresponding physical page number

Handling Page Faults and Page Tables

Physical memory is not permanently assigned to a specific process; instead, the operating system dynamically manages it using page tables. Each process has its own page table that maps its virtual pages to physical pages in RAM, creating the illusion of a private address space. The processor only generates virtual addresses, and the hardware (MMU) translates them using the current process's page table, ensuring that a process can only access the physical pages mapped to it. Other processes cannot access these pages because they have different mappings, and any illegal access is blocked by the hardware. Thus, isolation is achieved through OS-controlled mapping and hardware-enforced protection, not fixed ownership of physical memory. On special circumstances, two or more processes may share the physical page if the OS allows it - it might take place for shared libraries and shared memories etc.

- A page fault takes millions of clock cycles, so systems use fully associative placement to reduce fault frequency
- A page table resides in main memory to locate these pages without impractical full searches
- The page table is indexed by the virtual page number to discover the corresponding physical page number
- Each process has its own page table, which prevents virtual address spaces from colliding and enforces protection

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU
- Avoids going to DRAM for address translation on every access

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU
- Avoids going to DRAM for address translation on every access
- Typical capacity: 16–512 entries; usually fully associative

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU
- Avoids going to DRAM for address translation on every access
- Typical capacity: 16–512 entries; usually fully associative
- On every memory reference
  - TLB hit (∼99% of accesses): VPN → PPN returned in 1–2 cycles
  - TLB miss: hardware (or OS) must walk the page table in memory to reload the entry

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU
- Avoids going to DRAM for address translation on every access
- Typical capacity: 16–512 entries; usually fully associative
- On every memory reference
  - TLB hit (∼99% of accesses): VPN → PPN returned in 1–2 cycles
  - TLB miss: hardware (or OS) must walk the page table in memory to reload the entry
- On a context switch the TLB must be flushed (or tagged with an Address Space ID) to prevent one process from accessing another's pages

Translation Lookaside Buffer (TLB)

- A TLB is a small, fast cache of recently used page-table entries inside the CPU
- Avoids going to DRAM for address translation on every access
- Typical capacity: 16–512 entries; usually fully associative
- On every memory reference
  - TLB hit (∼99% of accesses): VPN → PPN returned in 1–2 cycles
  - TLB miss: hardware (or OS) must walk the page table in memory to reload the entry
- On a context switch the TLB must be flushed (or tagged with an Address Space ID) to prevent one process from accessing another's pages
- Combined lookup path: virtual address $\xrightarrow{\text{TLB}}$ physical address $\xrightarrow{\text{cache/DRAM}}$ data

Putting It All Together

- On a memory reference the CPU checks (in order) —
  1. TLB for address translation
  2. Cache for the data (using the now-physical address)
  3. DRAM if the cache misses
  4. Disk if a page fault occurs (OS intervenes)

Putting It All Together

- On a memory reference the CPU checks (in order) —
  1. TLB for address translation
  2. Cache for the data (using the now-physical address)
  3. DRAM if the cache misses
  4. Disk if a page fault occurs (OS intervenes)
- Virtual memory & cache: virtual memory manages the DRAM↔disk level; caches manage DRAM↔CPU levels

Putting It All Together

- On a memory reference the CPU checks (in order) —
  1. TLB for address translation
  2. Cache for the data (using the now-physical address)
  3. DRAM if the cache misses
  4. Disk if a page fault occurs (OS intervenes)
- Virtual memory & cache: virtual memory manages the DRAM↔disk level; caches manage DRAM↔CPU levels
- Protection at every level —
  - Page-table permission bits enforce read/write/execute rights
  - VMs add a second layer: the VMM controls which physical pages the guest OS may even see

Putting It All Together

- On a memory reference the CPU checks (in order) —
  1. TLB for address translation
  2. Cache for the data (using the now-physical address)
  3. DRAM if the cache misses
  4. Disk if a page fault occurs (OS intervenes)
- Virtual memory & cache: virtual memory manages the DRAM↔disk level; caches manage DRAM↔CPU levels
- Protection at every level —
  - Page-table permission bits enforce read/write/execute rights
  - VMs add a second layer: the VMM controls which physical pages the guest OS may even see
- *The OS course will cover page-replacement policies, multi-level page tables, demand paging, and more about VMs in depth*

Common Memory Hierarchy Framework

The Memory Hierarchy

- Common principles apply at all levels of memory hierarchy (based on our notions of caching)

The Memory Hierarchy

- Common principles apply at all levels of memory hierarchy (based on our notions of caching)
- Questions at each level in the hierarchy —
  1. Where Can a Block Be Placed?
  2. How Is a Block Found?
  3. Which Block Should Be Replaced on a Cache Miss?
  4. What Happens on a Write?

Block Placement

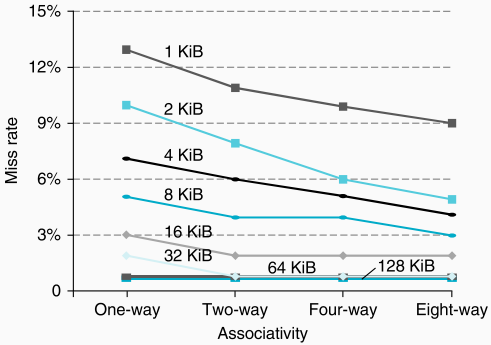
- Determined by associativity
  - Direct mapped (1-way associative): One choice for placement
  - *n*-way set associative: *n* choices within a set
  - Fully associative: Any location

Block Placement

- Determined by associativity
  - Direct mapped (1-way associative): One choice for placement
  - n-way set associative: n choices within a set
  - Fully associative: Any location
- Higher associativity reduces miss rate but increases complexity, cost, and access time

Block Placement

- Determined by associativity
  - Direct mapped (1-way associative): One choice for placement
  - *n*-way set associative: *n* choices within a set
  - Fully associative: Any location
- Higher associativity reduces miss rate but increases complexity, cost, and access time



Finding a Block

Associativity	Location Method	Tag Comparisons
Direct mapped	Index	1
n-way set associative	Set index, then search entries within set	n
Fully associative	Search all entries	Number of entries
	Full lookup table	0

Finding a Block

Associativity	Location Method	Tag Comparisons
Direct mapped	Index	1
n-way set associative	Set index, then search entries within set	n
Fully associative	Search all entries	Number of entries
	Full lookup table	0

- Hardware caches
  - Reduce comparisons to reduce cost

Finding a Block

Associativity	Location Method	Tag Comparisons
Direct mapped	Index	1
n-way set associative	Set index, then search entries within set	n
Fully associative	Search all entries	Number of entries
	Full lookup table	0

- Hardware caches
  - Reduce comparisons to reduce cost
- Virtual memory
  - Full table lookup makes fully associative feasible
  - Benefit in reduces miss rate

Replacement

- Choice of entry to replace on miss —

Replacement

- Choice of entry to replace on miss —
  - Least Recently Used (LRU): Complex and costly hardware for high associativity, often approximated in such cases

Replacement

- Choice of entry to replace on miss —
  - Least Recently Used (LRU): Complex and costly hardware for high associativity, often approximated in such cases
  - Random: Close to LRU for higher associativity, easier to implement

Replacement

- Choice of entry to replace on miss —
  - Least Recently Used (LRU): Complex and costly hardware for high associativity, often approximated in such cases
  - Random: Close to LRU for higher associativity, easier to implement
- Virtual memory can use LRU approximation with hardware support

Write Policy

- Write-through
  - Update both upper and lower levels
  - Simplifies replacement, but may require write buffer

Write Policy

- Write-through
  - Update both upper and lower levels
  - Simplifies replacement, but may require write buffer
- Write-back
  - Update upper level only
  - Update lower level when block is replaced
  - Need to keep track of more states

Write Policy

- Write-through
  - Update both upper and lower levels
  - Simplifies replacement, but may require write buffer
- Write-back
  - Update upper level only
  - Update lower level when block is replaced
  - Need to keep track of more states
- In the case of virtual memory, only write-back is feasible, given the disk write latency

The Three Cs: Intuitive Model for Sources of Misses

1. Compulsory misses (cold start misses)
   - First access to a block

The Three Cs: Intuitive Model for Sources of Misses

1. Compulsory misses (cold start misses)
   - First access to a block
2. Capacity misses
   - Due to the finite cache size
   - A replaced block is later accessed again

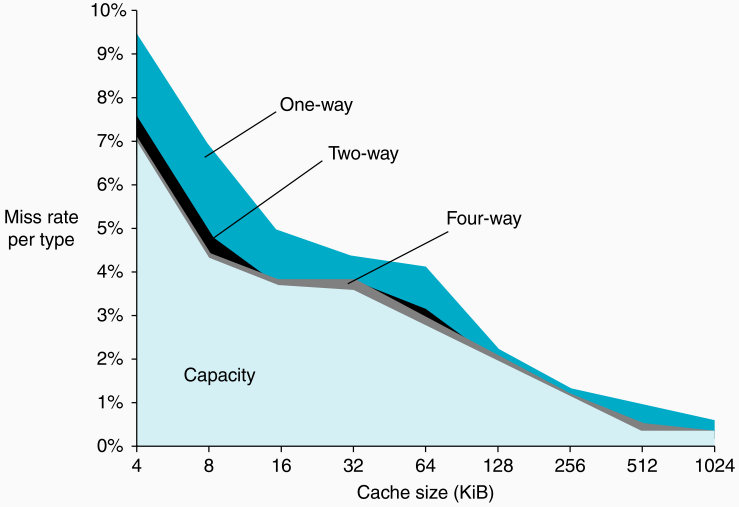
The Three Cs: Intuitive Model for Sources of Misses

1. Compulsory misses (cold start misses)
   - First access to a block
2. Capacity misses
   - Due to the finite cache size
   - A replaced block is later accessed again
3. Conflict misses (collision misses)
   - Happens in a non-fully associative cache
   - Due to competition of entries in a set
   - Would not occur in a fully associative cache of the same total size

Cache Design Trade-offs

Design Change	Effect on Miss Rate	Negative Performance Effect
Increase cache size	Decrease capacity miss	May increase access time
Increase associativity	Decreases miss rate due to conflict misses	May increase access time
Increase block size	Decreases miss rate for a wide range of block sizes due to spatial locality	Increases miss penalty; very large block could increase miss rate

Miss Rate Broken Down into Sources



Simple Cache Controller Using FSM

Cache Control

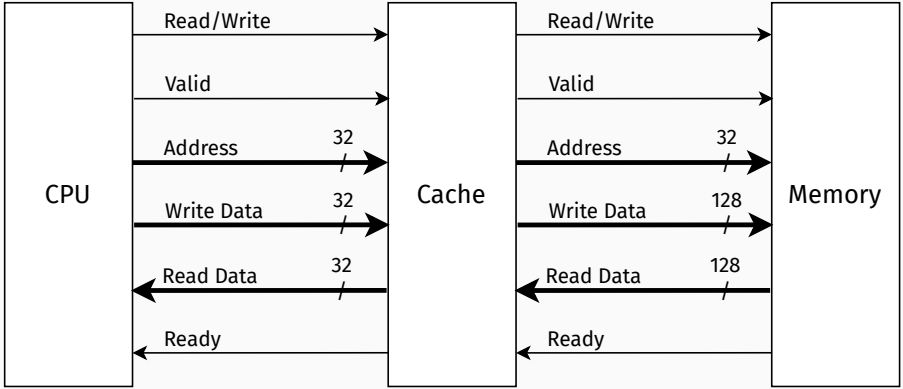
- Consider a simple cache with the following characteristics —
  - Direct-mapped, write-back, and write allocate
  - Block size: 4 words (16 bytes)
  - Cache size: 16 KB (1024 blocks)
  - 32-bit byte addressed
  - Valid bit and dirty bit per block
  - Blocking cache — CPU waits until access is complete

Cache Control

- Consider a simple cache with the following characteristics —
  - Direct-mapped, write-back, and write allocate
  - Block size: 4 words (16 bytes)
  - Cache size: 16 KB (1024 blocks)
  - 32-bit byte addressed
  - Valid bit and dirty bit per block
  - Blocking cache — CPU waits until access is complete

31 – 14	13 – 4	3 – 0
Tag	Index	Offset
18 bits	10 bits	4 bits

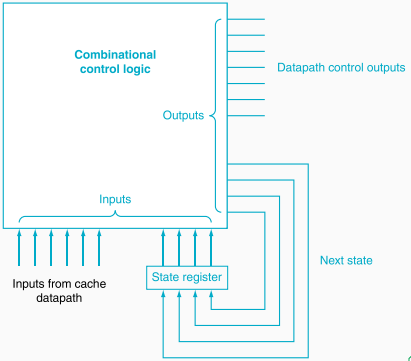
Interface Signals



Multiple cycles per access

Finite State Machines (FSMs)

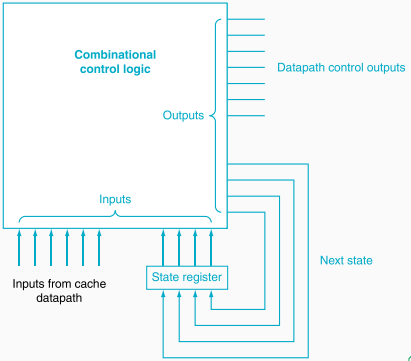
- We can use an FSM to sequence control steps



(Control signals in blue)

Finite State Machines (FSMs)

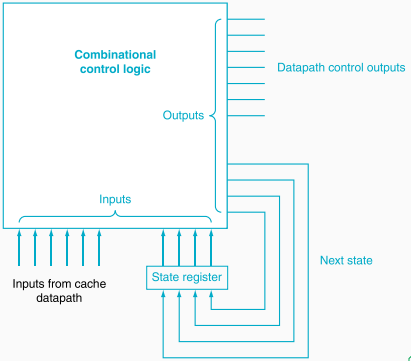
- We can use an FSM to sequence control steps
- Set of states to represent different 'modes' of operation



(Control signals in blue)

Finite State Machines (FSMs)

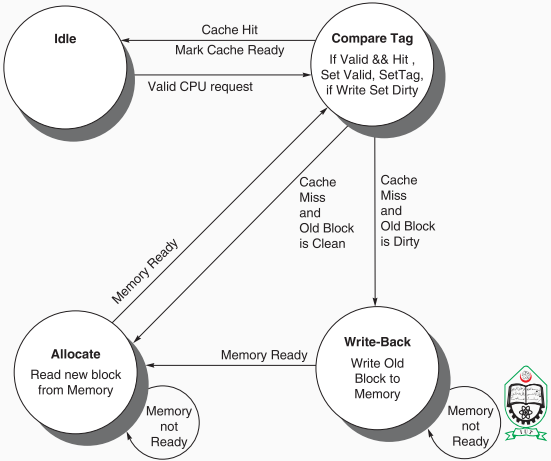
- We can use an FSM to sequence control steps
- Set of states to represent different 'modes' of operation
- Transition on each clock edge
  - State (S) values are binary encoded
  - Current state are stored in a register
  - Next state, $S_{i+1} = f_n(S_i, I_i)$, where I_i are the current inputs and f_n is a transition function
  - Control output signals, $C_i = g(S_i)$



(Control signals in blue)

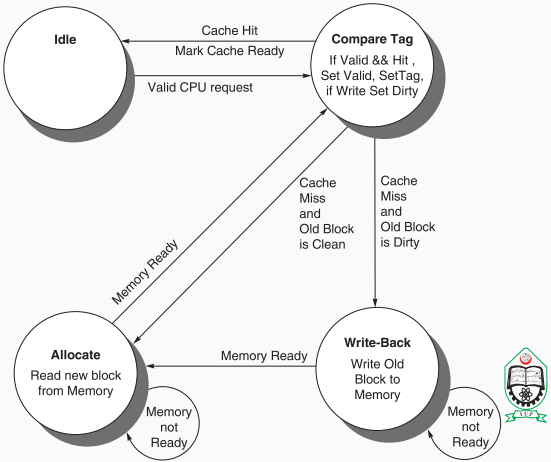
Cache Controller FSM

- Idle State
  - Waits for a valid read or write request from the processor
  - Transitions to the Compare Tag state once a request is received



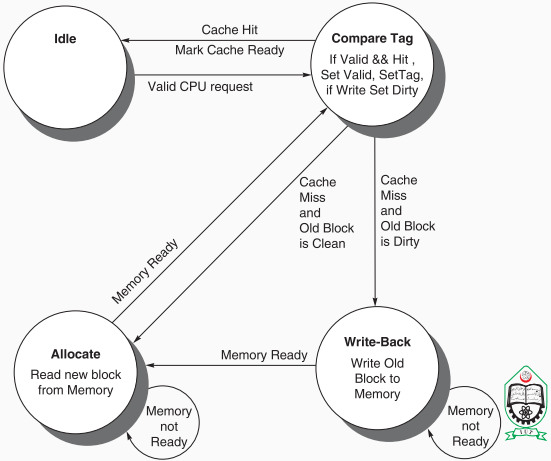
Cache Controller FSM

- Compare Tag State
  - Tests for cache hit or miss using the index portion of the address
  - Hit condition: Block is valid AND address tag matches cache tag
  - On read hit: Reads data from the selected word
  - On write hit: Writes to selected word and sets dirty bit, valid bit, and tag field
  - Sets the Cache Ready signal on a hit
  - Returns to Idle state after a successful hit
  - On miss: Updates cache tag and evaluates the dirty bit
  - Miss and dirty block: Transitions to Write-Back state
  - Miss and clean block: Transitions to Allocate state



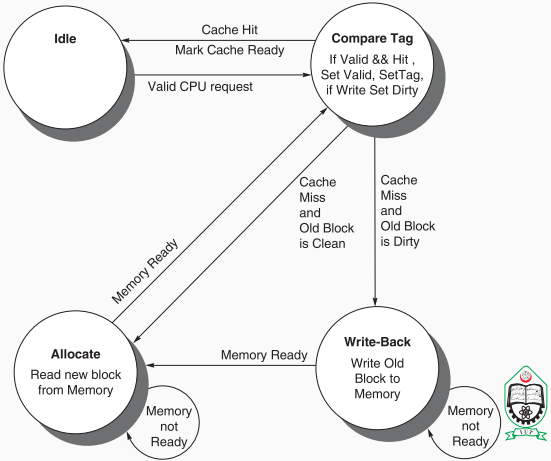
Cache Controller FSM

- Write-Back State
  - Writes the 128-bit block to memory
  - Uses address composed from the tag and cache index
  - Waits for the Ready signal from memory
  - Transitions to the Allocate state upon completion



Cache Controller FSM

- Allocate State
  - Fetches the new block from memory
  - Waits for the Ready signal from memory
  - Transitions to the Compare Tag state upon completion
  - Reuses Compare Tag state to overlap operations and update word on a write



Performance Extension of the FSM

- State Splitting
  - Simple model easily extends with more states to improve performance
  - Base model performs compare and cache access in a single clock cycle
  - Optimization: Separate compare and cache access into distinct states
  - Primary goal is to reduce the clock cycle time

Performance Extension of the FSM

state splitting - problem - compare and cache access take place in the same cycle which means the cycle must be long enough to support both of these operations.
fix - split that comparison and access parts into different cycles allowing the clock rate of the cpu to be reduced.
write buffer - problem -
for a dirty miss the existing cache line needs to be written back to the memory before new data can be read and evict the old entry, causing the cpu to wait for 2 mem accesses
fix - add a write buffer which where the dirty block is transferred to and the cpu can continue with the eviction immediately. the data is updated to the memory from the write buffer in the background

- State Splitting
  - Simple model easily extends with more states to improve performance
  - Base model performs compare and cache access in a single clock cycle
  - Optimization: Separate compare and cache access into distinct states
  - Primary goal is to reduce the clock cycle time
- Write Buffer
  - Optimization: Add a write buffer for dirty blocks
  - Saves the dirty block to the buffer instead of writing to memory immediately
  - Reads the new block first on a dirty miss
  - Prevents the processor from waiting for two full memory accesses
  - Cache writes the dirty block from the buffer while the processor resumes operation

Parallelism and Cache Coherence

Cache Coherence Problem

Suppose, two CPU cores share a physical address space, considering a write-through cache

Cache Coherence Problem

Suppose, two CPU cores share a physical address space, considering a write-through cache

Time Step	Event	CPU A's Cache	CPU B's cache	Memory (X)
0	–	–	–	0

Cache Coherence Problem

Suppose, two CPU cores share a physical address space, considering a write-through cache

Time Step	Event	CPU A's Cache	CPU B's cache	Memory (X)
0	–	–	–	0
1	CPU A reads X	0	–	0

Cache Coherence Problem

Suppose, two CPU cores share a physical address space, considering a write-through cache

Time Step	Event	CPU A's Cache	CPU B's cache	Memory (X)
0	–	–	–	0
1	CPU A reads X	0	–	0
2	CPU B reads X	0	0	0

Cache Coherence Problem

Suppose, two CPU cores share a physical address space, considering a write-through cache

Time Step	Event	CPU A's Cache	CPU B's cache	Memory (X)
0	–	–	–	0
1	CPU A reads X	0	–	0
2	CPU B reads X	0	0	0
3	CPU A writes 1 to X	1	0	1

Definition of Coherence

- Informally, reads return most recently written value

Definition of Coherence

- Informally, reads return most recently written value
- Formally,
  - P writes X; P reads X (no intervening writes) $\implies$ read returns written value
  - P_1 writes X; P_2 read X (sufficiently later) $\implies$ read returns written value
    - Contrary to CPU B reading X after step 3 in the previous example
  - P_1 writes X, P_2 writes X $\implies$ all processors see writes in the same order
    - End up with the same final value for X

Cache Coherence Protocols

- Protocols include operations performed by caches in multiprocessors to ensure cache coherence
  - Migration of data to local caches — reduces bandwidth for shared memory
  - Replication of read-shared data — reduces contention for access

Cache Coherence Protocols

- Protocols include operations performed by caches in multiprocessors to ensure cache coherence
  - Migration of data to local caches — reduces bandwidth for shared memory
  - Replication of read-shared data — reduces contention for access
- Snooping protocols
  - Each cache monitors a shared bus for reads/writes (invalidation)

Cache Coherence Protocols

- Protocols include operations performed by caches in multiprocessors to ensure cache coherence
  - Migration of data to local caches — reduces bandwidth for shared memory
  - Replication of read-shared data — reduces contention for access
- Snooping protocols
  - Each cache monitors a shared bus for reads/writes (invalidation)
- Directory-based protocols
  - Caches and memory record sharing status of blocks in a directory

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

CPU Activity	Bus Activity	CPU A's Cache	CPU B's cache	Memory (X)
–	–	–	–	0

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

CPU Activity	Bus Activity	CPU A's Cache	CPU B's cache	Memory (X)
–	–	–	–	0
CPU A reads X	Cache miss for X	0	–	0

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

CPU Activity	Bus Activity	CPU A's Cache	CPU B's cache	Memory (X)
–	–	–	–	0
CPU A reads X	Cache miss for X	0	–	0
CPU B reads X	Cache miss for X	0	0^S	0

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

CPU Activity	Bus Activity	CPU A's Cache	CPU B's cache	Memory (X)
–	–	–	–	0
CPU A reads X	Cache miss for X	0	–	0
CPU B reads X	Cache miss for X	0	0^s	0
CPU A writes 1 to X	Invalidate for X	1	$–^i$	0

Invalidating Snooping Protocols

- Cache gets exclusive access (ownership) to a block when it is to be written
  - Upon write, owner broadcasts an invalidate message on the bus
  - Subsequent read in another cache results in a miss; owning cache supplies the updated value

CPU Activity	Bus Activity	CPU A's Cache	CPU B's cache	Memory (X)
–	–	–	–	0
CPU A reads X	Cache miss for X	0	–	0
CPU B reads X	Cache miss for X	0	0^s	0
CPU A writes 1 to X	Invalidate for X	1	$–^i$	0
CPU B reads X	Cache miss for X	1	1^s	1^u

$\square^s$: snooped, $\square^i$: invalidated, $\square^u$: updated on share (simplification without ownership tracking)

Memory Consistency

- When are writes seen by other processors?
  - Seen means a read returns the written value
  - Cannot be instantaneous

Memory Consistency

- When are writes seen by other processors?
  - Seen means a read returns the written value
  - Cannot be instantaneous
- Assumptions —
  - A write is completed only when all processors have seen it
  - A processor does not reorder writes with other accesses

Memory Consistency

- When are writes seen by other processors?
  - Seen means a read returns the written value
  - Cannot be instantaneous
- Assumptions —
  - A write is completed only when all processors have seen it
  - A processor does not reorder writes with other accesses
- Consequence —
  - P writes X then writes $Y \implies$ all processors that see the new Y will also see the new X
  - Processors can reorder reads, but not writes

Conclusion

Pitfalls

- Bytes vs word addressing
  - Example: 32-byte direct-mapped cache, 4 byte block $\rightarrow$ byte 36 maps to block 1, but word 36 maps to block 4

Pitfalls

- Bytes vs word addressing
  - Example: 32-byte direct-mapped cache, 4 byte block → byte 36 maps to block 1, but word 36 maps to block 4
- Ignoring memory system effects when writing or generating code
  - Example: Iterating over rows vs columns of arrays — larger strides result in poor locality

Pitfalls

- Bytes vs word addressing
  - Example: 32-byte direct-mapped cache, 4 byte block → byte 36 maps to block 1, but word 36 maps to block 4
- Ignoring memory system effects when writing or generating code
  - Example: Iterating over rows vs columns of arrays — larger strides result in poor locality
- Using AMAT to evaluate performance of an out-of-order processor
  - Ignores effect of non-blocked addresses
  - Instead, evaluate performance by simulation

Pitfalls

- Bytes vs word addressing
  - Example: 32-byte direct-mapped cache, 4 byte block → byte 36 maps to block 1, but word 36 maps to block 4
- Ignoring memory system effects when writing or generating code
  - Example: Iterating over rows vs columns of arrays — larger strides result in poor locality
- Using AMAT to evaluate performance of an out-of-order processor
  - Ignores effect of non-blocked addresses
  - Instead, evaluate performance by simulation
- Implementing a VMM on an ISA not designed for virtualization
  - Example: Non-privileged instructions accessing hardware resources
  - Either extend the ISA or require guest OS not to use problematic instructions

Fallacies

- Disk failure rates in the field match their specifications
  - Independent study showed actual Annualized Failure Rates (AFR) of 2% to 4% against quoted 0.6% to 0.8%
  - Google study showed year-three AFRs reaching 8.6% against a quoted 1.5%
  - Field failure rates frequently measure three to six times higher than manufacturer declarations

Fallacies

- Disk failure rates in the field match their specifications
  - Independent study showed actual Annualized Failure Rates (AFR) of 2% to 4% against quoted 0.6% to 0.8%
  - Google study showed year-three AFRs reaching 8.6% against a quoted 1.5%
  - Field failure rates frequently measure three to six times higher than manufacturer declarations
- Operating systems are the best place to schedule disk accesses
  - OS only accesses logical block addresses and attempts to optimize by sorting them in increasing order
  - Disks utilize the actual mapping of physical geometry including sectors, tracks, and surfaces
  - Disk-level rescheduling actively minimizes rotational and seek latencies
  - OS reordering can inflate completion times to multiple revolutions compared to fractions of a revolution under disk scheduling

Concluding Remarks

- Fast memories are small, large memories are slow
  - We really want fast, large memories ☹
  - Caching gives this illusion ☺

Concluding Remarks

- Fast memories are small, large memories are slow
  - We really want fast, large memories ☹
  - Caching gives this illusion ☺
- Principle of locality (spatial and temporal)
  - Programs use a small part (often contiguous) of their memory space frequently

Concluding Remarks

- Fast memories are small, large memories are slow
  - We really want fast, large memories ☹
  - Caching gives this illusion ☺
- Principle of locality (spatial and temporal)
  - Programs use a small part (often contiguous) of their memory space frequently
- Memory hierarchy

$$L_1 \text{ cache} \leftrightarrow L_2 \text{ cache} \leftrightarrow \cdots \leftrightarrow \text{DRAM Memory} \leftrightarrow \text{Secondary Storage}$$

Concluding Remarks

- Fast memories are small, large memories are slow
  - We really want fast, large memories ☹
  - Caching gives this illusion ☺
- Principle of locality (spatial and temporal)
  - Programs use a small part (often contiguous) of their memory space frequently
- Memory hierarchy

$$L_1 \text{ cache} \leftrightarrow L_2 \text{ cache} \leftrightarrow \cdots \leftrightarrow \text{DRAM Memory} \leftrightarrow \text{Secondary Storage}$$

- Coherence and Consistency
  - Proper memory system design is critical for multi-core and multi-processor systems

Thank You!